

Projeto: EatZ

Equipe:

Arthur Garcia – RM 364139

Kauã Leal – RM 363862

Luíza Rosa – RM 363912

1. Introdução

Descrição do problema

Restaurantes de pequeno e médio porte enfrentam dificuldades na gestão de pedidos, no relacionamento com clientes e na presença digital eficiente. Muitos não possuem um sistema próprio, ou utilizam soluções limitadas que não atendem às suas necessidades operacionais, como controle de cardápio, registro de pedidos e avaliação de clientes.

O objetivo é criar um sistema robusto que permita a todos os restaurantes gerenciar eficientemente suas operações, enquanto os clientes poderão consultar informações, deixar avaliações e fazer pedidos online. Esse sistema também permitirá que os clientes escolham restaurantes com base na comida oferecida, em vez de se basearem na qualidade do sistema de gestão.

Devido à limitação de recursos financeiros, foi acordado que o desenvolvimento será realizado de forma incremental, com entregas divididas em fases, garantindo que cada etapa seja desenvolvida de forma cuidadosa e eficaz. A aplicação deverá ser escalável, segura e bem estruturada, garantindo uma base sólida para futuras evoluções e integrações.

Objetivo do projeto

O objetivo principal deste projeto é desenvolver uma API back-end robusta utilizando o framework Spring Boot para gerenciar diferentes tipos de usuários e atender aos requisitos definidos.

2. Arquitetura do Sistema

O sistema back-end do projeto **Eatz** segue uma arquitetura limpa e modularizada, baseada nos princípios do DDD (Domain-Driven Design) e na separação de responsabilidades. A estrutura do projeto é dividida em camadas bem definidas:

Camadas da Arquitetura

- **Domain:** Contém as entidades principais do negócio (Customer, Address,

RestaurantUser) e seus contratos (interfaces de repositórios) e exceções específicas.

- **Application:** Implementa os casos de uso de negócio, como criação, autenticação, atualização e remoção de usuários. Cada lógica é encapsulada em uma classe com um único propósito, como CreateCustomerUseCase ou AuthenticateRestaurantUserUseCase.
- **Infrastructure:** Contém a persistência de dados (JPA Repositories e entidades do banco), configuração de segurança (JWT), Swagger, e tratamento global de exceções (GlobalExceptionHandler).
- **Presentation (web):** Responsável pela interface HTTP (Controllers), DTOs de requisição e resposta. Embora os controllers em si não estejam listados no trecho analisado, é presumível sua presença pela estrutura das camadas.

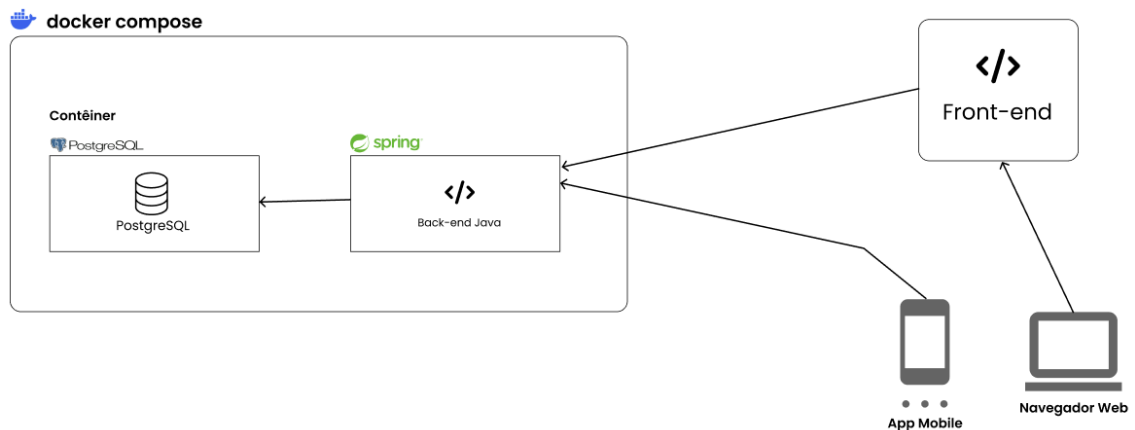
Tecnologias Utilizadas

- **Spring Boot 3** – Framework principal para a API REST
- **JWT (JSON Web Token)** – Para autenticação de clientes e restaurantes
- **Spring Security** – Para proteção dos endpoints
- **PostgreSQL** – Banco de dados relacional
- **JPA/Hibernate** – ORM para persistência dos dados
- **Docker + Docker Compose** – Containerização e orquestração local
- **Swagger/OpenAPI** – Documentação interativa da API
- **Postman** – Testes manuais de endpoints

Segurança

- O sistema protege as rotas com autenticação JWT, diferenciando os usuários por tipo (CUSTOMER, RESTAURANT_USER) e papel (ADMIN, EMPLOYEE, etc.).
- Tokens expirados ou revogados são rejeitados por meio de um serviço de blacklist.
- Handlers personalizados (JwtAccessDeniedHandler, JwtAuthenticationEntryPoint) tratam erros de autenticação de forma padronizada.

Diagrama da Arquitetura



3. Descrição dos Endpoints da API

Tabela de Endpoints

Endpoint	Método	Descrição
/customers	GET	Busca um cliente pela validação do token
/customers/{id}	GET	Busca um cliente por ID
/customers/register	POST	Cadastra um novo usuário do tipo cliente
/customers/login	POST	Retorna o token de autenticação do usuário
/customers/{customerId}/address	POST	Adiciona um novo endereço
/customers/{id}	PUT	Atualiza informações gerais do usuário
/customers/{customerId}/address/{id}	PUT	Atualiza informações de um endereço
/customers/password	PATCH	Atualiza a senha do usuário
/customers/{id}	DELETE	Remove um usuário
/customers/{customerId}/address/{addressId}	DELETE	Remove um endereço do usuário
/restaurant-users	GET	Busca um funcionário pela validação do token
/restaurant-users/{id}	GET	Busca um funcionário por ID
/restaurant-users/register	POST	Cadastra um novo usuário do tipo funcionário

/restaurant-users/login	POST	Retorna o token de autenticação do usuário
/restaurant-users/{id}	PUT	Atualiza informações gerais e de endereço do usuário
/restaurant-users/password	PATCH	Atualiza a senha do usuário
/restaurant-users/{id}	DELETE	Remove um usuário
/restaurants/{id}	GET	Busca um restaurante por ID
/restaurants	POST	Cadastra um restaurante
/restaurants/{id}	PUT	Atualiza um restaurante
/restaurants/{id}	DELETE	Remove um restaurante
/restaurant-user-types/all	GET	Busca todos os tipos cadastrados
/restaurant-user-types/{id}	GET	Busca um tipo por ID
/restaurant-user-types	POST	Cadastra um novo tipo
/restaurant-user-types/{id}	PUT	Atualiza um tipo
/restaurant-user-types/{id}	DELETE	Remove um tipo
/menu-items/{id}	GET	Busca um item do menu por ID
/menu-items/restaurant/{restaurantID}	GET	Busca os itens do menu de um restaurante
/menu-items	POST	Cadastra um item do menu
/menu-items/{id}	PUT	Atualiza um item do menu
/menu-items/{id}	DELETE	Remove um item do menu

Exemplos de requisição e resposta

1. Clientes

Endpoint: GET /customers

Exemplo de requisição: /customers

Requer Autenticação

Exemplo de resposta

Status: 200 OK

```
{
  "addresses": [
    {
      "id": 1,
      "street": "Rua das Laranjeiras, 124",
      "number": "455",
      "complement": null,
      "neighborhood": "Centro",
      "city": "Rio de Janeiro",
      "state": "RJ",
    }
  ]
}
```

```
        "zipCode": "43210-567",
        "nickname": "Casa",
        "default": true
    }
],
    "id": 1,
    "name": "Sarah Rosa Oliveira",
    "email": "sarah.rosa2@email.com",
    "phone": "980807071",
    "cpf": "35582555506",
    "profileImageUrl": null,
    "createdAt": "2025-05-26T23:59:19.729329",
    "updatedAt": "2025-05-26T23:59:42.988418"
}
```

Endpoint: GET /customers/{id}

Exemplo de requisição: /customers/1

Requer Autenticação

Exemplo de resposta

Status: 200 OK

```
{
  "addresses": [
    {
      "id": 1,
      "street": "Rua das Laranjeiras, 124",
      "number": "455",
      "complement": null,
      "neighborhood": "Centro",
      "city": "Rio de Janeiro",
      "state": "RJ",
      "zipCode": "43210-567",
      "nickname": "Casa",
      "default": true
    }
  ],
  "id": 1,
  "name": "Sarah Rosa Oliveira",
  "email": "sarah.rosa2@email.com",
  "phone": "980807071",
  "cpf": "35582555506",
  "profileImageUrl": null,
  "createdAt": "2025-05-26T23:59:19.729329",
  "updatedAt": "2025-05-26T23:59:42.988418"
}
```

Endpoint: POST /customers/register

Exemplo de requisição:

```
{
  "name": "Sarah Rosa Silva",
  "email": "sarah.rosa2@email.com",
```

```
"password": "senha*123",
"phone": "980807070",
"cpf": "87688855501",
"profileImageUrl": "www.image-url.com",
"address": {
  "street": "Rua das Laranjeiras, 124",
  "number": "455",
  "complement": null,
  "neighbourhood": "Centro",
  "city": "Rio de Janeiro",
  "state": "RJ",
  "zipCode": "43210-567",
  "nickname": "Casa",
  "defaultAddress": true
}
}
```

Exemplo de resposta:

Status: 201 CREATED

```
{
  "addresses": [
    {
      "id": 1,
      "street": "Rua das Laranjeiras, 124",
      "number": "455",
      "complement": null,
      "neighborhood": "Centro",
      "city": "Rio de Janeiro",
      "state": "RJ",
      "zipCode": "43210-567",
      "nickname": "Casa",
      "default": true
    }
  ],
  "id": 1,
  "name": "Sarah Rosa Silva",
  "email": "sarah.rosa2@email.com",
  "phone": "980807070",
  "cpf": "87688855501",
  "profileImageUrl": "www.image-url.com",
  "createdAt": "2025-05-26T23:47:50.9195978",
  "updatedAt": null
}
```

Endpoint: POST /customers/login

Exemplo de requisição:

```
{
  "email": "joao.silva@email.com",
  "password": "senha*123",
}
```

Exemplo de resposta:

Status: 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJmMjM0NTY3ODkwlwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjOnRydWU",
  "type": "Bearer",
  "expiresAt": "2025-04-25T12:00:00.000+00:00",
  "username": "joao.silva@email.com"
}
```

Endpoint: POST /customers/{id}/address

Exemplo de requisição: /customers/1/address

Requer Autenticação

```
{
  "street": "Rua das Figueiras",
  "number": "111",
  "complement": "11º andar",
  "neighbourhood": "Centro",
  "city": "Rio de Janeiro",
  "state": "RJ",
  "zipCode": "43210-567",
  "nickname": "Trabalho",
  "defaultAddress": false
}
```

Exemplo de resposta:

Status: 204 NO CONTENT

Endpoint: PATCH /customers/password

Exemplo de requisição:

Requer Autenticação

```
{
  "oldPassword": "senha*123",
  "newPassword": "senha#321",
}
```

Exemplo de resposta:

Status: 204 NO CONTENT

Endpoint: PUT /customers/{id}

Exemplo de requisição: /customers/1

Requer Autenticação

```
{
  "name": "Sarah Rosa Oliveira",
  "email": "sarah.rosa2@email.com",
  "phone": "980807071",
  "cpf": "35582555506",
  "profileImageUrl": null
}
```

Exemplo de resposta:

Status: 200 OK

```
{
  "id": 1,
  "name": " Sarah Rosa Oliveira",
  "email": " sarah.rosa2@email.com",
  "phone": "980807071",
  "cpf": "35582555506",
  "profileImageUrl": null,
  "createdAt": "2025-05-22T00:37:23.344538200",
  "updatedAt": "2025-05-22T00:42:51.757884400",
  "addresses": []
}
```

Endpoint: PUT /customers/{customerId}/address/{addressId}

Exemplo de requisição: /customers/1/address/1

Requer Autenticação

```
{
  "street": "Rua das Palmeiras",
  "number": "222",
  "complement": "Apto 22",
  "neighbourhood": "Vila Velha",
  "city": "São Paulo",
  "state": "SP",
  "zipCode": "02123-111",
  "nickname": "Apto",
  "defaultAddress": true
}
```

Exemplo de resposta:

Status: 204 NO CONTENT

Endpoint: DELETE /customers/{id}

Exemplo de requisição: /customers/1

Requer Autenticação

Exemplo de resposta:

Status: 204 NO CONTENT

Endpoint: DELETE /customers/{customerId}/address/{addressId}

Exemplo de requisição: /customers/1/address/1

Requer Autenticação

Exemplo de resposta:

Status: 204 NO CONTENT

2. Funcionários

Endpoint: GET /restaurant-users

Requer Autenticação

Exemplo de resposta

Status: 200 OK


```
{
  "id": 1,
  "name": "João Funcionário",
  "email": "joao.ferreira@email.com",
  "cpf": "88877799901",
  "phone": "970708080",
  "role": "ADMIN",
  "profileImageUrl": "www.image-url.com",
  "restaurantId": null,
  "createdAt": "2025-05-27T00:21:41.915538",
  "updatedAt": null,
  "address": {
    "id": 3,
    "street": "Rua das Floriculturas",
    "number": "42",
    "complement": "222B",
    "neighborhood": "Centro",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-567"
  }
}
```

Endpoint: GET /restaurant-users/{id}

Exemplo de requisição: /restaurant-users /1

Requer Autenticação

Exemplo de resposta

Status: 200 OK

```
{
  "id": 1,
  "name": "João Funcionário",
  "email": "joao.ferreira@email.com",
  "cpf": "88877799901",
  "phone": "970708080",
  "role": "ADMIN",
  "profileImageUrl": "www.image-url.com",
  "restaurantId": null,
  "createdAt": "2025-05-27T00:21:41.915538",
  "updatedAt": null,
  "address": {
    "id": 3,
    "street": "Rua das Floriculturas",
    "number": "42",
    "complement": "222B",
    "neighborhood": "Centro",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-567"
  }
}
```

Endpoint: POST /restaurant-users/register

Exemplo de requisição:

```
{
  "name": "João Funcionário",
  "email": "joao.ferreira@email.com",
  "password": "senha*123",
  "role": "ADMIN",
  "cpf": "88877799901",
  "phone": "970708080",
  "profileImageUrl": "www.image-url.com",
  "address": {
    "street": "Rua das Floriculturas",
    "number": "42",
    "complement": "222B",
    "neighbourhood": "Centro",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-567"
  }
}
```

Exemplo de resposta:

Status: 201 CREATED

```
{
  "id": 1,
  "name": "João Funcionário",
  "email": "joao.ferreira@email.com",
  "cpf": "88877799901",
  "phone": "970708080",
  "role": "ADMIN",
  "profileImageUrl": "www.image-url.com",
  "restaurantId": null,
  "createdAt": "2025-05-27T00:21:41.9155382",
  "updatedAt": null,
  "address": {
    "id": 3,
    "street": "Rua das Floriculturas",
    "number": "42",
    "complement": "222B",
    "neighborhood": "Centro",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-567"
  }
}
```

Endpoint: POST /restaurant-users/login

Exemplo de requisição:

```
{
  "email": "joao.ferreira@email.com",
  "password": "senha*123",
}
```

```
}
```

Exemplo de resposta:

Status: 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJmM0NTY3ODkwiibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0iOnRydWUsImhhdCI6MTUxNjIzOTAyMn0.KMUFsIDTnFmyG3nMiGM6H9FNFUOf3wh7SmqJp-QV30",
  "type": "Bearer",
  "expiresAt": "2025-04-25T12:00:00.000+00:00",
  "username": "joao.ferreira@email.com"
}
```

Endpoint: PATCH /restaurant-users/password

Exemplo de requisição:

Requer Autenticação

```
{
  "oldPassword": "senha*123",
  "newPassword": "senha#321",
}
```

Exemplo de resposta:

Status: 204 NO CONTENT

Endpoint: PUT /restaurant-users/{id}

Exemplo de requisição: /restaurant-users/1

Requer Autenticação

```
{
  "name": "João Silva Marques",
  "email": "joao.ferreira@email.com",
  "role": "MANAGER",
  "cpf": "88877799902",
  "phone": "970708082",
  "profileImageUrl": "www.image-url.com",
  "address": {
    "street": "Rua das Libélulas",
    "number": "24",
    "complement": "111A",
    "neighbourhood": "Vila Velha",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-569"
  }
}
```

Exemplo de resposta:

Status: 200 OK

```
{
  "id": 1,
  "name": "João Silva Marques",
  "email": "joao.ferreira@email.com",
  "cpf": "88877799902",
  "phone": "970708082",
  "role": "MANAGER",
}
```

```
"profileImageUrl": "www.image-url.com",
"restaurantId": null,
"createdAt": "2025-05-27T00:21:41.915538",
"updatedAt": "2025-05-27T00:38:33.8332004",
"address": {
  "id": 3,
  "street": "Rua das Libélulas",
  "number": "24",
  "complement": "111A",
  "neighborhood": "Vila Velha",
  "city": "São Paulo",
  "state": "SP",
  "zipCode": "01234-569"
}
}
```

Endpoint: DELETE /restaurant-users/{id}

Exemplo de requisição: /restaurant-users/1

Requer Autenticação

Exemplo de resposta:

Status: 204 NO CONTENT

3. Restaurante

Endpoint: POST /restaurants

Descrição: Cadastra um novo restaurante com endereço associado

Requer Autenticação

Exemplo de requisição:

```
{
  "name": "Restaurante do João",
  "logolImageUrl": "https://meusite.com/imagens/restaurante-joao.png",
  "phone": "11999999999",
  "whatsappPhone": "11988888888",
  "cnpj": "12.345.678/0001-99",
  "deliveryRadius": 5.0,
  "ownerId": 1,
  "address": {
    "street": "Rua das Oliveiras",
    "number": "123",
    "complement": "Casa 2",
    "neighbourhood": "Jardim das Árvores",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-000"
  }
}
```

Exemplo de resposta:

Status: 200 OK

```
{
```

```
"id": 10,
"name": "Restaurante do João",
"logoImageUrl": "https://meusite.com/imagens/restaurante-joao.png",
"phone": "11999999999",
"whatsappPhone": "11988888888",
"cnpj": "12.345.678/0001-99",
"deliveryRadius": 5.0,
"createdAt": "2025-08-05T13:00:00.000Z",
"updatedAt": null,
"address": {
  "id": 22,
  "street": "Rua das Oliveiras",
  "number": "123",
  "complement": "Casa 2",
  "neighborhood": "Jardim das Árvores",
  "city": "São Paulo",
  "state": "SP",
  "zipCode": "01234-000"
}
}
```

Endpoint: GET /restaurants/{id}

Descrição: Retorna os dados de um restaurante específico

Requer Autenticação

Exemplo de requisição: /restaurants/10

Exemplo de resposta:

Status: 200 OK

```
{
  "id": 10,
  "name": "Restaurante do João",
  "logoImageUrl": "https://meusite.com/imagens/restaurante-joao.png",
  "phone": "11999999999",
  "whatsappPhone": "11988888888",
  "cnpj": "12.345.678/0001-99",
  "deliveryRadius": 5.0,
  "createdAt": "2025-08-05T13:00:00.000Z",
  "updatedAt": null,
  "address": {
    "id": 22,
    "street": "Rua das Oliveiras",
    "number": "123",
    "complement": "Casa 2",
    "neighborhood": "Jardim das Árvores",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-000"
  }
}
```

Endpoint: PUT /restaurants/{id}

Descrição: Atualiza os dados de um restaurante

Requer Autenticação

Exemplo de requisição: /restaurants/10

```
{
  "name": "Restaurante Gourmet do João",
  "logolImageUrl": "https://meusite.com/imagens/gourmet.png",
  "phone": "11999999999",
  "whatsappPhone": "11988888888",
  "cnpj": "12.345.678/0001-99",
  "deliveryRadius": 7.5,
  "ownerId": 1,
  "address": {
    "street": "Rua das Palmeiras",
    "number": "321",
    "complement": "Apto 5",
    "neighbourhood": "Centro",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-999"
  }
}
```

Exemplo de resposta:

Status: 200 OK

```
{
  "id": 10,
  "name": "Restaurante Gourmet do João",
  "logolImageUrl": "https://meusite.com/imagens/gourmet.png",
  "phone": "11999999999",
  "whatsappPhone": "11988888888",
  "cnpj": "12.345.678/0001-99",
  "deliveryRadius": 7.5,
  "createdAt": "2025-08-05T13:00:00.000Z",
  "updatedAt": "2025-08-06T12:00:00.000Z",
  "address": {
    "id": 22,
    "street": "Rua das Palmeiras",
    "number": "321",
    "complement": "Apto 5",
    "neighborhood": "Centro",
    "city": "São Paulo",
    "state": "SP",
    "zipCode": "01234-999"
  }
}
```

Endpoint: DELETE /restaurants/{id}

Descrição: Exclui logicamente um restaurante

Requer Autenticação

Exemplo de requisição: /restaurants/10

Exemplo de resposta:

Status: 204 NO CONTENT

4. Tipo de funcionário

Endpoint: GET /restaurant-user-types/all

Descrição: Retorna todos os tipos de funcionário cadastrados.

Exemplo de resposta:

Status: 200 OK

```
[
  {
    "id": 1,
    "name": "Gerente",
    "description": "Responsável pela gestão do restaurante",
    "createdAt": "2025-01-01T10:00:00",
    "updatedAt": "2025-01-01T10:00:00"
  }
]
```

Endpoint: GET /restaurant-user-types/{id}

Descrição: Retorna um tipo de funcionário com base no ID.

Exemplo de requisição: /restaurant-user-types/1

Requer Autenticação

Exemplo de resposta:

Status: 200 OK

```
{
  "id": 1,
  "name": "Gerente",
  "description": "Responsável pela gestão do restaurante",
  "createdAt": "2025-01-01T10:00:00",
  "updatedAt": "2025-01-01T10:00:00"
}
```

Endpoint: POST /restaurant-user-types

Descrição: Cria um tipo de funcionário.

Requer Autenticação

Exemplo de requisição:

```
{
  "name": "Garçom",
  "description": "Responsável pelo atendimento às mesas"
}
```

Exemplo de resposta:

Status: 201 Created

```
{
  "id": 2,
  "name": "Garçom",
  "description": "Responsável pelo atendimento às mesas",
  "createdAt": "2025-01-02T11:00:00",
  "updatedAt": "2025-01-02T11:00:00"
}
```

```
}
```

Endpoint: PUT /restaurant-user-types/{id}

Descrição: Atualiza um tipo de funcionário existente.

Requer Autenticação

Exemplo de requisição: /restaurant-user-types/1

```
{
  "name": "Garçom",
  "description": "Atendimento aos clientes"
}
```

Exemplo de resposta:

Status: 200 OK

```
{
  "id": 2,
  "name": "Garçom",
  "description": "Atendimento aos clientes",
  "createdAt": "2025-01-02T11:00:00",
  "updatedAt": "2025-01-03T14:00:00"
}
```

Endpoint: DELETE /restaurant-user-types/{id}

Descrição: Remove um tipo de funcionário com base no ID.

Exemplo de requisição: /restaurant-user-types /1

Requer Autenticação

Status: 204 No Content

5. Item do menu

Endpoint: POST /menu-items

Descrição: Cadastra um novo item no cardápio vinculado a um restaurante

Requer Autenticação

Exemplo de requisição:

```
{
  "name": "Pizza Margherita",
  "description": "Pizza com molho de tomate, mussarela e manjericão",
  "price": 39.90,
  "onlyLocalConsumption": false,
  "photoBase64": "dados_em_base64",
  "photoContentType": "image/png",
  "restaurantId": 10
}
```

Exemplo de resposta:

Status: 201 CREATED

```
{
  "id": 101,
  "name": "Pizza Margherita",
  "description": "Pizza com molho de tomate, mussarela e manjericão",
  "price": 39.90,

```



```
"onlyLocalConsumption": false,  
"photoUrl": "https://meusite.com/fotos/pizza-margherita.png",  
"restaurantId": 10,  
"createdAt": "2025-08-05T15:30:00",  
"updatedAt": null  
}
```

Endpoint: GET /menu-items/{id}

Descrição: Retorna os dados de um item de menu específico

Requer Autenticação

Exemplo de requisição: /menu-items/1

Exemplo de resposta:

Status: 200 OK

```
{  
  "id": 101,  
  "name": "Pizza Margherita",  
  "description": "Pizza com molho de tomate, mussarela e manjericão",  
  "price": 39.90,  
  "onlyLocalConsumption": false,  
  "photoUrl": "https://meusite.com/fotos/pizza-margherita.png",  
  "restaurantId": 10,  
  "createdAt": "2025-08-05T15:30:00",  
  "updatedAt": null  
}
```

Endpoint: GET /menu-items/restaurant/{restaurantId}

Descrição: Retorna todos os itens do menu de um restaurante específico

Requer Autenticação

Exemplo de requisição: /menu-items/restaurant/1

Exemplo de resposta:

Status: 200 OK

```
[  
  {  
    "id": 101,  
    "name": "Pizza Margherita",  
    "description": "Pizza com molho de tomate, mussarela e manjericão",  
    "price": 39.90,  
    "onlyLocalConsumption": false,  
    "photoUrl": "https://meusite.com/fotos/pizza-margherita.png",  
    "restaurantId": 10,  
    "createdAt": "2025-08-05T15:30:00",  
    "updatedAt": null  
  },  
  {  
    "id": 102,  
    "name": "Lasanha de Frango",  
    "description": "Lasanha caseira com frango desfiado e molho branco",  
    "price": 29.90,  
    "onlyLocalConsumption": true,  
  }  
]
```

```
"photoUrl": "https://meusite.com/fotos/lasanha.png",
"restaurantId": 10,
"createdAt": "2025-08-05T15:45:00",
"updatedAt": null
}
]
```

Endpoint: PUT /menu-items/{id}

Descrição: Atualiza os dados de um item de menu existente

Requer Autenticação

Exemplo de requisição: /menu-items/1

```
{
  "name": "Pizza Marguerita Especial",
  "description": "Versão especial com borda recheada",
  "price": 45.00,
  "onlyLocalConsumption": false,
  "photoBase64": "novos_dados_base64",
  "photoContentType": "image/jpeg",
  "restaurantId": 10
}
```

Exemplo de resposta:

Status: 200 OK

```
{
  "id": 101,
  "name": "Pizza Marguerita Especial",
  "description": "Versão especial com borda recheada",
  "price": 45.00,
  "onlyLocalConsumption": false,
  "photoUrl": "https://meusite.com/fotos/pizza-marguerita-especial.jpg",
  "restaurantId": 10,
  "createdAt": "2025-08-05T15:30:00",
  "updatedAt": "2025-08-06T10:15:00"
}
```

Endpoint: DELETE /menu-items/{id}

Descrição: Remove logicamente um item do menu com base no ID

Requer Autenticação

Exemplo de requisição: /menu-items/1

Exemplo de resposta:

Status: 204 NO CONTENT

6. Erros

401 Unauthorized

Uso: para login e token inválido em endpoints autenticados.

Exemplo de resposta:

```
{
  "path": "/error",
}
```

```
"error": "Acesso não autorizado",
"message": "Full authentication is required to access this resource",
"status": 401
}
```

404 Not Found

Uso: para recursos não encontrados.

Exemplo de resposta:

```
{
  "path": "/customers/1/address/42",
  "error": "Not Found",
  "message": "Endereço não encontrado na lista de endereços do cliente",
  "timestamp": "2025-05-26T20:49:33.13903",
  "status": 404
}
```

409 Conflict

Uso: para cadastro de usuários duplicados.

Exemplo de resposta:

```
{
  "path": "/customers/register",
  "error": "Conflict",
  "message": "Já existe um usuário com este e-mail.",
  "timestamp": "2025-05-26T00:01:28.7160215",
  "status": 409
}
```

4. Configuração do Projeto

Configuração do Docker Compose

O Docker Compose atua como um orquestrador local, facilitando a configuração e execução conjunta de múltiplos containers que formam a aplicação — neste caso, a API Java Spring e o banco de dados PostgreSQL, centralizando as definições necessárias.

Possui dois serviços principais:

- **db:** Banco de dados PostgreSQL
 - Usa a imagem oficial postgres:latest;
 - Configura o banco de dados (register_db);
 - Expõe a porta padrão do PostgreSQL: 5432.
- **api:** API Java Spring
 - É construído localmente a partir do Dockerfile localizado no projeto.

- Conecta-se ao banco de dados via URL interna (jdbc:postgresql://db:5432/register_db), utilizando as credenciais configuradas.
- Expõe a porta da aplicação: 8080.
- O serviço api depende do db, ou seja, o banco de dados será iniciado antes da API.

Instruções para execução local

1. Pré-requisitos

Para rodar este projeto, é necessário ter instalado:

- Java 21
- Docker e Docker Compose
- Git (opcional, para clonar o repositório)

2. Clonando o Projeto

Execute o comando abaixo para clonar o repositório e acessar a pasta do projeto:

```
git clone https://github.com/EatzFiap/register-api.git
cd register-api
```

3. Executando com Docker Compose

Execute o comando abaixo para subir a aplicação e o banco de dados:

```
docker-compose up --build
```

Esse comando irá:

- Compilar e executar a aplicação Spring Boot.
- Iniciar um container PostgreSQL com as credenciais apropriadas.
- Expor a API na porta 8080.

4. Acessos Disponíveis

- API: <http://localhost:8080>
- Swagger (documentação interativa): <http://localhost:8080/swagger-ui.html>

5. Configuração do Banco de Dados

O banco de dados PostgreSQL será iniciado automaticamente via Docker. Seguem as configurações:

- Host: localhost
- Porta: 5432
- Usuário: local
- Senha: local123
- Database: register_db

Essas credenciais estão configuradas no docker-compose.yml e como variáveis de ambiente da aplicação.

5. Qualidade do Código

Boas Práticas Utilizadas

Durante o desenvolvimento da API, diversas boas práticas foram adotadas para garantir legibilidade, manutenção e escalabilidade do código:

Separação de Responsabilidades (SRP - SOLID): A lógica da aplicação foi dividida em camadas bem definidas (controllers, use cases, services e DTOs), favorecendo o princípio da responsabilidade única.

Padrões RESTful: Os endpoints seguem convenções REST, utilizando corretamente os verbos HTTP (GET, POST, PUT, DELETE) e nomes de recursos (/customers, /restaurant-users, etc.), o que facilita o entendimento e integração por parte de desenvolvedores externos.

Reutilização de Código: Classes como CustomerAddress estendem Address, evitando duplicação de atributos comuns. Além disso, objetos como AddressRequest são utilizados para encapsular e validar entradas.

Validações com Bean Validation: Anotações como @NotBlank foram aplicadas diretamente nos DTOs para garantir a integridade dos dados de entrada.

Clareza e padronização nos nomes: Os nomes de classes, métodos e pacotes seguem uma convenção clara e autoexplicativa, em conformidade com os padrões do Spring Boot.

Modularização e desacoplamento: Casos de uso como getCustomerUseCase encapsulam a lógica de negócio, facilitando testes unitários e evitando acoplamento direto com os controladores.

6. Collections para Teste

Collection do Postman

[Link para collection](#)

Outra forma de acessar a collection é através do repositório do projeto no Github. Na pasta “docs” é possível encontrar os arquivos para importar no Postman ou em outro client que aceite arquivos do Postman. Importe a collection pelo arquivo Register API.postman_collection e o environment Local.postman_environment para utilizar as variáveis locais.

Descrição dos Testes Manuais

Para validar os endpoints manualmente através do Postman, é necessário primeiramente selecionar o recurso a ser testado, pois as requisições estão agrupadas com base nesse critério. Por exemplo, as requisições relacionadas a clientes estão na pasta customers.

As requisições de GET são para realizar busca de dados, POST para cadastro, PUT e PATCH para atualização de dados e DELETE para remoção de cadastros.

Apenas as requisições de criação de usuário e de login não requerem autenticação através do token retornado no login. Em algumas requisições GET e DELETE, é necessário passar um ID através da URL especificando que cadastro será manipulado. Então para buscar o usuário com ID 12, a URL deve conter /customers/12.

Nas requisições POST, PUT e PATCH, é possível alterar as informações que estão sendo enviadas através da aba body, em formato de JSON.

Para testar as requisições sendo enviadas, basta escolher o ambiente (local, desenvolvimento, produção) e clicar em enviar. O endpoint retornará o status de sucesso ou de erro condizente com a requisição.

7. Repositório do Código

URL do Repositório no GitHub

<https://github.com/EatzFiap/register-api>